How the optimiser works, from the basics to the choices

A companion note to the v7 campaign report — consolidated edition

C. Leclercq E. Clément

September 7, 2026 — revision 7

[Consolidated edition: eight rounds of review questions (August 27 – September 4) have been folded into the exposition. The order is deliberate — what a constrained optimiser is (§2–§3), then the minimax direction problem solved by hand (§4), then the three methods of the literature on their own terms (§5), then their comparison (§6), then our method as shipped (§7), then the choices ahead (§8–§9). Everything introduced is defined before it is used. Passages rewritten in the seventh revision (the Mengi group's optimise-and-enrich framework and its consequences for the inner solver, the margin constraint and the position cases) are set in this colour. The oracle (values, gradients, peaks, certificates) is taken as given; its algebra is the companion note.]

1 The problem, and why one word makes it hard

Collect everything we may tune — controller coefficients, gain, actuator and sensor parameters — into one vector $\theta \in \mathbb{R}^{n_\theta}$. For every frequency ω the oracle returns the closed-loop amplification $f_\omega(\theta)$, an ordinary differentiable function of θ (with one caveat met in §7.5). We want to minimise the worst case,

$$f(\theta) = \max_{\omega} f_\omega(\theta),$$

subject to closed-loop stability with a margin ε (§7.4).

Everything difficult is in the word **max**. Push the tallest peak down and another rises; push until they tie, and from then on any move that lowers one raises the other. The minimiser of a maximum sits, generically, at a *kink*: several peaks are level, "which one is the maximum" switches there, and the gradient jumps. Plain gradient descent computes the gradient of whichever peak is tallest, steps, finds another tallest, steps back, and zigzags across the crease. **The optimum is a balance of peaks, and every method in this note is a way of computing balances.** One more thing to fix in mind from the start: there are two kinds of kink in this problem. A kink *between* frequencies — several peaks tie at the top — is the one every method below is built for. A kink *inside* one frequency — at some ω the two largest singular values of the response coincide, so f_ω itself switches branch — is one level down, invisible to the outer balance, and it is where the shipped method still fails (§7.5). Apart from the stability constraint, these two kinks are the whole of the nonsmoothness.

Two words used throughout. Every method alternates between two levels. The *inner* level works on a finite list of frequencies $\Omega = \{\omega_1, \ldots, \omega_N\}$ (the *working set*) and moves θ using only the oracle's values and gradients there. The *outer* level consults the oracle over the whole band — certifies the true maximum, locates peaks the list missed — and updates the list. Because there are infinitely many frequencies and finitely many unknowns, the problem is a *semi-infinite program*, and "solve on a finite set, add the violated points, repeat" is its classical solution pattern — the *exchange* method (Hettich & Kortanek, *SIAM Review* 1993), whose ancestor is Remez's algorithm for best polynomial approximation (equalise the error on a reference set, exchange in the point where it is largest). Apkarian and Noll call the same move *enrichment*.

2 Constrained optimisation, from zero

This section is the toolbox. It is written for a reader who knows what a gradient is and nothing else, and every later section uses it.

2.1 Lagrangian, multipliers, KKT, active set

Consider $\min_x F(x)$ s.t. $c_i(x) \le 0$, $i = 1,\dots,m$, all smooth. Attach to every constraint a nonnegative number λ_i, its *multiplier*, and form the *Lagrangian*

$$\mathscr{L}(x,\lambda) = F(x) + \sum_i \lambda_i c_i(x).$$

Read λ_i as the *price* of constraint i: how much the optimal value would improve per unit of relaxation of $c_i \le 0$. At a solution x^* there exist prices $\lambda^* \ge 0$ such that (the *KKT conditions*)

$$\nabla F(x^*) + \sum_i \lambda_i^* \nabla c_i(x^*) = 0, \qquad c_i(x^*) \le 0, \qquad \lambda_i^* c_i(x^*) = 0.$$

The first says: $-\nabla F$ is a nonnegative combination of the gradients of the binding constraints — one cannot descend without pushing into a wall. The third (*complementarity*) says a constraint is either binding ($c_i = 0$) or free ($\lambda_i = 0$), never both priced and slack. The binding constraints form the *active set* $\mathscr{A}$. Off $\mathscr{A}$ the inactive constraints can be deleted without changing anything locally — which is why every active-set method *guesses* the active set, solves with equalities, and checks the signs; interior point methods (§2.3) are the one family that never guesses, and they pay for it differently.

Primal and dual. The original problem and its variable x are the *primal*. For fixed prices $\lambda \ge 0$, minimise the Lagrangian over x freely: $q(\lambda) = \min_x \mathscr{L}(x,\lambda)$, the *dual function*. Since $\mathscr{L}(x,\lambda) \le F(x)$ whenever x is feasible ($c_i \le 0$, $\lambda_i \ge 0$), $q(\lambda) \le F(x)$ for every feasible x: any price vector gives a certified lower bound (*weak duality*). The *dual problem* is the best bound, $\max_{\lambda \ge 0} q(\lambda)$; its solution is the KKT multiplier vector, and for the convex problems in this note (all the *quadratic programs*, QPs — quadratic objective, linear constraints — below) the bound is tight: $\max q = \min F$ (*strong duality*). "Dual variables", "multipliers", "prices" are three names for λ. A *primal–dual* method updates x and λ together.

The curvature that matters. For a constrained problem the relevant second-order object is the Hessian of the Lagrangian, $\nabla^2_{xx}\mathscr{L} = \nabla^2 F + \sum_i \lambda_i \nabla^2 c_i$: curvature of the objective *and* of the binding constraints, price-weighted.

2.2 The SQP model problem, derived two ways

Fix the active set (so complementarity is resolved). The KKT conditions become a square system of equations in $(x, \lambda_{\mathscr{A}})$: $R(x,\lambda) = \left[\nabla F + \sum_{\mathscr{A}} \lambda_i \nabla c_i;\ c_{\mathscr{A}}\right] = 0$, where $\nabla c_{\mathscr{A}}$ denotes the matrix whose rows are the active constraint gradients.

Route 1, Newton on KKT. Newton linearises R and solves one linear system:

$$\boxed{\begin{bmatrix} \nabla^2_{xx}\mathscr{L} & \nabla c_{\mathscr{A}}^\top \\ \nabla c_{\mathscr{A}} & 0 \end{bmatrix} \begin{bmatrix} d \\ \lambda^+ - \lambda \end{bmatrix} = - \begin{bmatrix} \nabla_x \mathscr{L} \\ c_{\mathscr{A}} \end{bmatrix}}$$

Written out, its two block rows are $\nabla^2_{xx}\mathscr{L}\, d + \nabla c_{\mathscr{A}}^\top \lambda^+ = -\nabla F$ and $c_{\mathscr{A}} + \nabla c_{\mathscr{A}} d = 0$.

Route 2, Taylor. Expand the Lagrangian to second order in x at fixed λ and the constraints to first order, and minimise the quadratic model of the Lagrangian subject to the affine (linear-plus-constant)

model of the constraints. On the linearised feasible set $\nabla c_i^\top d = -c_i$ is a constant, so the term $\sum \lambda_i \nabla c_i^\top d$ of the model is a constant and can be dropped; what remains is

$$\boxed{\min_d \ \nabla F^\top d + \tfrac{1}{2} d^\top \left(\nabla^2_{xx}\mathscr{L}\right) d \qquad \text{s.t.} \qquad c_i + \nabla c_i^\top d \le 0.}$$

Check that the routes agree. Write this QP's own Lagrangian, $\ell(d,\mu) = \nabla F^\top d + \frac{1}{2} d^\top \nabla^2_{xx}\mathscr{L}\, d + \sum \mu_i (c_i + \nabla c_i^\top d)$, and set its first variation to zero: in d, $\nabla F + \nabla^2_{xx}\mathscr{L}\, d + \sum \mu_i \nabla c_i = 0$; in μ, $c_i + \nabla c_i^\top d = 0$. These are the two block rows above with $\mu = \lambda^+$: the QP's own multipliers are the Newton update of the prices. (Had the dropped constant been kept, the QP would return $\mu = \lambda^+ - \lambda$ instead, same d — the old λ enters only through $\nabla^2_{xx}\mathscr{L}$.) So "solve the QP" and "take the Newton step" are one operation; the QP form exists because, when the active set is not known, inequalities and multiplier signs must be managed, which a linear system cannot express and a QP handles natively. *Sequential quadratic programming* (SQP) iterates this model problem, with the exact Hessian replaced by an estimate B and a step control (§2.4).

2.3 How a QP is solved: active set, interior point, ADMM

Three families, each used somewhere in this note. One distinction comes first. A QP can arise in two ways: as *the subproblem of an SQP step* (the model problem above, solved once per iteration of an outer NLP loop), or as *the whole problem*. And an interior point method can be applied at two places: to the QP inside an SQP step (then it is a *QP solver*, one option among the three below), or directly to the NLP (then there is no SQP loop and no QP at all — each iteration is one linear system on the perturbed KKT conditions, §3; that is what IPOPT does). "Level 1" in §8 means the choice of NLP method (SQP versus IPM-on-the-NLP); "level 2" means, if SQP, which QP solver sits inside.

Active set — guess which inequalities are binding, solve the equality-constrained problem (one linear system), check the two tests (every multiplier nonnegative; no excluded constraint violated), and walk: drop a constraint whose multiplier came out negative, add one that was trampled, one small solve per change. Exact, superb warm starts (the previous active set is usually right), cost proportional to the number of changes — combinatorial in the worst case. The hand method of §4; qpOASES, `quadprog`.

Interior point — never guess the active set: keep every inequality strictly satisfied by adding a barrier $-\mu \sum \log(-c_i)$ to the objective, take Newton steps, and let $\mu \to 0$ (§3). The iterates never reach the active set; they approach it from inside until the distance is below tolerance, which for every practical purpose is the same answer — the difference from the active-set walk is not in the answer but in the road: no combinatorial guessing, at the price of poor warm starts on the boundary. Robust, insensitive to the number of constraints. IPOPT at the NLP level; Gurobi/MOSEK for QPs.

ADMM (alternating direction method of multipliers) — write the inequality constraints $l \le Ax \le u$ of the QP by introducing a copy $z = Ax$ that must lie in the box $l \le z \le u$ (the box *is* the inequalities), and form the augmented Lagrangian of the equality $Ax = z$,

$$L_\rho(x,z,y) = \tfrac{1}{2} x^\top P x + q^\top x + y^\top (Ax - z) + \tfrac{\rho}{2} \|Ax - z\|^2,$$

with multiplier y and penalty ρ. Then alternate three cheap steps: minimise L_ρ over x (one linear system with matrix $P + \rho A^\top A$, factorised once); minimise over z (clip $Ax + y/\rho$ into the box, entry by entry); update $y \leftarrow y + \rho(Ax - z)$. No active set is ever guessed — the clip decides, entry by entry, which bounds are hit at each iteration. Each iteration is a triangular solve and a clip; hundreds of iterations to modest accuracy (10^{-3}–10^{-4}), warm starts natural, scales to large problems. OSQP — which is what the frozen pipeline uses inside PyGRANSO (`o.QPsolver='osqp'`), a fact that matters in §8.

2.4 Step control: line search, then trust region

A model proposes a direction d (and, for a QP with curvature, a length: the full step $\beta = 1$). Reality is consulted along the ray $x + \beta d$ by a *line search*. An *exact* line search would minimise $\varphi(\beta) = F(x + \beta d)$

over β — never done, since every trial is an oracle call; every line search in this note is *inexact*: it stops at the first β that passes a test. The basic test is *Armijo*: try $\beta = 1$, accept if the true function decreased by at least a fraction of what the model promised, $\varphi(\beta) \le \varphi(0) + c_1\beta\varphi'(0)$ with $c_1 \sim 0.1$; else halve and retry. A first-order method needs nothing more.

The curvature condition for quasi-Newton. A quasi-Newton method builds its curvature estimate B from the pairs $s = x^+ - x$, $y = \nabla\mathscr{L}^+ - \nabla\mathscr{L}$ by the BFGS update $B \leftarrow B - \frac{Bss^\top B}{s^\top Bs} + \frac{yy^\top}{y^\top s}$, which keeps B positive definite only if $y^\top s > 0$; the second *Wolfe* condition $\varphi'(\beta) \ge c_2\varphi'(0)$ (the slope has become less negative — "weak"; "strong" Wolfe bounds $|\varphi'(\beta)|$ from both sides) guarantees it. So: first-order → Armijo; quasi-Newton → Armijo plus curvature condition.

Trust region. A *trust region* is the other step-control idea: cap $\|d\| \le \Delta$ inside the model, then accept or reject the whole step (never shrink along the ray) and adapt Δ from the ratio of actual to predicted decrease. Note that a quadratic term $\frac{1}{2}\|d\|^2/\tau$ in a direction problem is *not* by itself a trust region: by Lagrangian duality it is the same as the cap $\|d\| \le \Delta(\tau)$, so it fixes the *scale* of the step, and what makes a method a trust-region method is what it does *next* — line-search methods take the direction and shrink along it (the descent line, GRANSO, our inner solver); a trust-region method keeps or rejects the whole step and adapts τ (the bundle line, whose proximity control is a trust region wearing a penalty; and our outer step governor, §7.4).

2.5 Two facts about nonsmooth functions

Two facts belong here, because one method of §5 rests on them and the kinks of §1 make them relevant to all.

BFGS on a nonsmooth function (Lewis & Overton 2013). A kinked function is differentiable almost everywhere, so an inexact line search that never lands exactly on a kink (probability zero for an Armijo–Wolfe search with random-looking trial lengths) always has a gradient to use. The secant pairs taken across a kink encode a jump of the gradient as a huge curvature, so B becomes wildly ill-conditioned — and, minimising a nonsmooth *objective* directly, that is useful: the large curvature is across the crease, so the steps align with the crease and follow it, at a linear rate, in practice to a nonsmooth minimiser. The theory covers special cases only; when the method fails it stalls silently, and it eventually stagnates at the accuracy the line search can resolve. This is the behaviour HIFOO (§5.2) exploits.

Gradient sampling (Burke, Lewis & Overton 2005). At the current point sample $m > n_\theta$ random points in a ball of radius ε ($n_\theta = \dim\theta$; the count is set by the dimension of the space, not by the number of active pieces, which the method does not know), compute their gradients, take the minimum-norm element of their convex hull as the direction (the weight problem of §4.2 again), line-search, and shrink ε when that norm is small — an estimate of the Clarke subdifferential by *random sampling* instead of by enumeration of the active pieces. Advantages: convergence theory (to Clarke stationary points, with probability one) for any locally Lipschitz function, and a rough optimality measure for free (the smallest hull norm). Limits: first-order, m gradients per iteration, no curvature — an endgame tool, not an engine.

2.6 Acronyms and terms

QP quadratic program (quadratic objective, linear constraints). **NLP** nonlinear program. **SQP** sequential quadratic programming. **IPM** interior point method. **SDP** semidefinite program — the LMI route to H_∞ synthesis, which squares the number of unknowns and does not reach our scales; nothing here solves one. **Hard/soft** requirement: enforced / minimised as far as possible (standard usage; Apkarian's tools adopt it). **Epigraph**: the region on or above a graph; "epigraph reformulation" = give a maximum its own variable and constrain every member below it (§7.1; used constraint by constraint in §3). **Subdifferential**: at a kink a function has no gradient but a *set* of slopes; for a convex function it is the set of slopes

of the planes lying below the graph, and for a nonconvex one the *Clarke* subdifferential generalises it as the convex hull of the limits of nearby gradients — for a maximum of smooth functions, the convex hull of the active gradients. The two notions coincide for convex functions. Every convergence theorem in this note is stated in this language ("Clarke stationary" = zero belongs to the subdifferential), but no algorithm in this note ever *computes* a subdifferential. The subdifferential of a maximum is an infinite set (a convex hull) generated by finitely many vectors — the gradients of the active pieces — and every method works with those finitely many *generators* only: the descent line as the columns of its weight problem, the bundle as its planes, our SQP as its constraint rows. The minimax direction of §4.1 is the minimum-norm element of the hull, computed from the generators; the cutting-plane model of §5.3 is the hull re-assembled from generators collected at several points. So the Clarke subdifferential is relevant to the theory of all three treatments — it is the object the convergence theorems are stated for — while the implementations manipulate its generators.

3 Four ways to handle a constraint: quadratic penalty, exact penalty, barrier, progress function

Every constrained method must discourage steps that violate constraints. Four devices are in use in this note; write $[c]_+ = \max(c,0)$ for a violation:

$$\begin{aligned}
&\text{quadratic penalty:} && P_\rho(x) = F + \tfrac{\rho}{2}\sum_i [c_i]_+^2, && \rho \to \infty;\\
&\text{exact penalty:} && \phi_\mu(x) = \mu F + \sum_i [c_i]_+, && \mu \text{ small enough;}\\
&\text{log barrier:} && B_\mu(x) = F - \mu \sum_i \log(-c_i), && \mu \to 0;\\
&\text{progress function:} && \Phi_\eta(x) = \max\{F-\eta,\ c_1,\dots,c_m\}, && \text{bisect on } \eta.
\end{aligned}$$

The first two allow violation and charge for it from outside; the barrier forbids it, with a fence that becomes infinite at the boundary; the fourth, the *progress function*, puts objective and constraints on one footing — every term is a "must be ≤ 0" test once the objective is shifted by a target level η — and searches for the smallest achievable η by bisection.

The whole difference on one line of real numbers. Minimise $F(x) = x$ subject to $x \ge 1$ ($c = 1 - x \le 0$): solution $x^* = 1$, price $\lambda^* = 1$.

- Quadratic penalty $x + \frac{\rho}{2}[1-x]_+^2$: for $x < 1$ the slope is $1 - \rho(1-x)$, zero at $x_\rho = 1 - 1/\rho$. *Infeasible for every finite* ρ; exact only in the limit. Its price $\rho[1-x_\rho]_+ = 1$ is right only because the violation stays λ^*/ρ.
- Barrier $x - \mu\log(x-1)$: slope $1 - \mu/(x-1)$, zero at $x_\mu = 1 + \mu$. *Strictly feasible for every* $\mu > 0$; exact only in the limit. Its second derivative at the minimiser is $1/\mu$ — the *ill-conditioning* of barriers: the bowl becomes infinitely steep as $\mu \to 0$, and Newton systems with an entry $\sim 1/\mu$ lose $\log_{10}(1/\mu)$ digits.
- Exact penalty $\mu x + [1-x]_+$: for $x < 1$ the slope is $\mu - 1$, *negative* if $\mu < 1$; for $x > 1$ it is $\mu > 0$. The minimiser is *exactly* $x = 1$ for every $\mu \in (0,1)$: the constraint is held by the *kink*, not by a limit. The condition $\mu < 1 = 1/\lambda^*$ says the penalty must be able to pay the price; it can pay at most $1/\mu$.
- Progress function $\max\{x-\eta,\ 1-x\}$: the two lines cross at $x = (1+\eta)/2$, where the minimum value is $(1-\eta)/2$. It is ≤ 0 exactly when $\eta \ge 1$, so the smallest achievable level is $\eta = 1$, at which the minimiser is $x = 1$ — the constrained optimum, *exactly*, found by bisection on η (each trial being one unconstrained minimisation of a kinked function).

Quadratic penalty and barrier hold a constraint by a limit; the exact penalty holds it by a kink. A kink needs no limit, which is why any small enough μ works and none is unique.

	quadratic penalty	exact penalty	log barrier	progress function
smooth?	C^1	kink at every $c_i = 0$	yes, where defined	kink wherever two terms tie
exact for finite parameter?	never	yes, once $\mu < 1/\max \lambda_i^*$ (Han 1977)	never (central path)	yes, at one unique η: the optimal value
iterates	slightly infeasible	either side	strictly inside	either side
Hessian conditioning	blows up as $\rho \to \infty$	bounded	λ^{*2}/μ: blows up (cured by the primal–dual form, §3.1)	bounded
warm start on the boundary	fine	fine	awkward	fine
feasible start needed?	no	no	yes (or slacks)	no
used here as	(merit functions only)	GRANSO's model and merit (§5.4)	IPOPT (§8)	systune (§5.3)

A penalty serves two jobs a method may assign separately: the *model* whose minimisation proposes a step, and the *merit function* that decides whether to accept it. GRANSO uses the exact penalty for both; plain SQP uses explicit constraints as model and an ℓ_1 merit; IPOPT uses the barrier as model and a filter.

3.1 The step problem each one generates

Replace F, c_i by their local models ($\nabla F^\top d + \frac{1}{2} d^\top B d$, $c_i + \nabla c_i^\top d$) inside the penalised function and minimise over d.

Quadratic penalty: a piecewise-quadratic unconstrained problem whose stationarity condition is a linear system once one knows which linearised constraints are violated — the same guess-and-check as an active set; price $\lambda_i = \rho\,[c_i + \nabla c_i^\top d]_+$, proportional to the violation.

Exact penalty: a QP via slacks — the epigraph trick, constraint by constraint. The step problem $\min_d \mu \nabla f^\top d + \frac{1}{2} d^\top B d + \sum_i [c_i + \nabla c_i^\top d]_+$ is nonsmooth. Use $[a]_+ = \min\{s :\ s \geq a,\ s \geq 0\}$, one variable s_i per term:

$$\min_{d,s}\ \mu \nabla f^\top d + \tfrac{1}{2} d^\top B d + \sum_i s_i \qquad \text{s.t.} \qquad s_i \geq c_i + \nabla c_i^\top d,\ \ s_i \geq 0.$$

At the solution each s_i sits on the larger of its bounds, so this QP has the same minimiser in d as the nonsmooth problem: the nonsmoothness has migrated into constraints. Its multipliers: Lagrangian $\ell = \mu \nabla f^\top d + \frac{1}{2} d^\top B d + \sum s_i + \sum \nu_i (c_i + \nabla c_i^\top d - s_i) - \sum \xi_i s_i$ with $\nu, \xi \geq 0$; the ∂_{s_i} condition gives $\nu_i + \xi_i = 1$, hence $0 \leq \nu_i \leq 1$. Three cases: violated at the solution ($s_i > 0$): $\xi_i = 0$, $\nu_i = 1$ — charged the full unit price, whatever the violation; strictly satisfied: $\nu_i = 0$; exactly active: $\nu_i \in [0,1]$ from the balance. Dividing ∂_d by μ identifies the true multipliers as $\lambda_i = \nu_i/\mu \leq 1/\mu$: *the penalty can pay at most* $1/\mu$ *per constraint*. If every $\lambda_i^* < 1/\mu$, all slacks vanish and the penalty QP *is* the SQP model problem of §2.2 (Han 1977); if some $\lambda_i^* \geq 1/\mu$, that constraint is left violated and μ must decrease.

Barrier: a linear system. Newton on B_μ directly has the $1/\mu$ conditioning above. Instead name the price $z_i := -\mu/c_i$ and write stationarity as $\nabla F + \sum z_i \nabla c_i = 0$, $-z_i c_i = \mu$: the KKT conditions with complementarity relaxed from 0 to μ. Newton on this system in (x, z) is the *primal–dual* step; the ill-conditioning is confined to the active block, where it is benign. Three basic facts: μ never reaches zero, so no constraint is ever exactly active during the run; "active" is decided by comparing two numbers — constraint i is treated as active when $-c_i \leq \tau$ and $z_i > \tau$, with τ tied to the final μ (e.g. $\sqrt{\mu}$, since $-z_i c_i = \mu$); and the *central path* is simply the curve of minimisers $\mu \mapsto (x_\mu, z_\mu)$, ending at (x^*, λ^*) — in the one-dimensional example, the points $(1+\mu,\ 1)$.

3.2 Driving the parameter

Exact penalty — steering (Curtis, Mitchell & Overton 2017). Let $v(d) = \sum_i [c_i + \nabla c_i^\top d]_+$ be the linearised violation after a step. μ starts at 1 and only decreases. Each iteration: (1) *probe*: solve the QP with the objective switched off ($\mu = 0$), obtain d_0 and $\Delta_{\max} = v(0) - v(d_0)$, the largest violation decrease any step can buy; (2) solve the QP with the current μ, obtain d_μ and $\Delta_\mu = v(0) - v(d_\mu)$; (3) **test** $\Delta_\mu \geq c_v \Delta_{\max}$, $c_v = 0.1$; (4) if it holds keep μ and take d_μ, else $\mu \leftarrow \frac{1}{2}\mu$ and go to (2). It terminates because $d_\mu \to d_0$ as $\mu \to 0$. (Steering is due to Byrd, Nocedal & Waltz 2008; GRANSO adopts it with a monotone μ.) GRANSO never lets μ regrow. Two remedies are standard elsewhere: resetting the penalty parameter at every outer restart — which the frozen pipeline *does*, since each exchange round calls PyGRANSO afresh with $\mu = 1$, so any decay is confined to one inner solve and cannot accumulate across rounds — and increasing it by a fixed factor when a run has been feasible for many iterations while the objective decrease per iteration has become negligible relative to $\|\nabla F\|$ (the ℓ_1-penalty literature, Nocedal & Wright §18.3). The second is a two-line detector, and it is part of to-do item 0. A caveat for us. Exactness holds for every $\mu < 1/\max \lambda_i^*$, but a *much* smaller μ is not free: μ multiplies the objective, so with $\mu = 10^{-6}$ the merit function ϕ_μ is a million times more sensitive to a unit of violation than to a unit of objective, and a line search on it accepts steps for their feasibility while barely registering progress in F. Since every halving of μ is permanent, a halving triggered by a *temporary* violation — a step across a kink, an iterate that momentarily breaks many constraints — leaves μ far below what the prices require for the rest of the run. Exact, yes; well-conditioned, no. This is a plausible mechanism behind the "inner at its limit" terminations of the campaign, and it can be read directly from the logged value of μ at the stalled iterates; the cure, if it is the cause, is the reset-at-restart or regrow rule above — a solver setting, not a change of method.

Barrier — schedule and monitor (IPOPT). Residuals at (x, z): $r_d = \nabla F + \sum z_i \nabla c_i$ (dual: balance), $r_p = [c]_+$ (primal: violation), $r_c = (-z_i c_i - \mu)_i$ (complementarity: distance to the central path); $E_\mu =$ the largest entry of the three (dual and complementarity scaled by $s_d, s_c \geq 1$ when multipliers are huge). Inner loop: Newton steps until $E_\mu \leq 10\mu$. Outer: $\mu \leftarrow \max\{\varepsilon_{\text{tol}}/10, \min(0.2\mu, \mu^{1.5})\}$ — linear far away, superlinear near the end — until $E_0 \leq \varepsilon_{\text{tol}}$. For us the feasible start is free (raise t; start from a stable design); the warm start after an exchange round is the awkward part (§8).

Progress function versus exact penalty. The two are close relatives: both replace the constraints by a max-type function and minimise it without constraints. The difference is what balances objective against constraints. The exact penalty has one weight, μ, trading the objective against *all* violations at once — so constraints of different physical units (a gain in one, a damping rate in another) must be pre-weighted by hand before a single μ can mean anything; steering then adapts μ within one run. The progress function balances by *dividing each requirement by its own target* and shifting by the common level η. The targets are the user's specifications — systune's "tuning goals" are each written as $F_k(\theta)/\gamma_k \leq 1$, with γ_k the gain, damping or bandwidth the designer asks for (for a soft objective whose achievable value is unknown, any positive γ_k will do: η absorbs the choice). Constraints are divided by a fixed positive scale of their own, never by their current value, so no division by zero arises. Then $\Phi_\eta = \max_k \{F_k/\gamma_k - \eta,\ c_i\}$: every term is dimensionless, and $\Phi_\eta \leq 0$ means "every requirement is met to within the factor η of its target" — $\eta = 1$ is "all targets met", $\eta = 0.8$ is "all beaten by 20%". The same division can of course be applied before an exact penalty; the difference is that the progress function *requires* it and makes η interpretable, whereas the penalty does not require it and, without it, gives μ no meaning across units. That is why systune uses it for multi-requirement tuning. The price is the search on η: bisection costs 5–10 warm-started runs, whereas steering settles μ inside one run — steering converges faster in solve count, bisection needs no weights and cannot be driven to a badly conditioned μ. For a single objective in one unit, as here, the exact penalty (or explicit constraints) loses nothing.

4 The direction problem solved by hand — the baseline for everything

Stand at θ with the peaks that are (nearly) tied with the maximum — the *active peaks* at $\omega_1, \dots, \omega_m$, with gradients $g_i = \nabla_\theta f_{\omega_i}(\theta)$. Because the maximum follows its tallest member, the local model is $f(\theta + d) \approx f(\theta) + \max_i g_i^\top d$. A good direction makes every $g_i^\top d$ negative at once. To rank directions, look at each through the eyes of its *worst* active peak (the largest $g_i^\top d$ — a max over i) and choose the direction whose worst outcome is smallest (a min over d). A linear model cannot say how far to trust itself, so the comparison must be at fixed length:

$$d^* = \arg\min_{\|d\| \le 1} \max_i \; g_i^\top d.$$

(The ball $\|d\| \le 1$ rather than the sphere: the ball is convex, which the duality below needs; nothing is lost, because a descent direction of length < 1 can be scaled up to length 1 and do better, so the minimiser sits on the sphere unless it is $d = 0$ — the case where no direction helps, which the ball lets us represent.)

4.1 Duality, gently: swap the players

Think of a game: you pick d, an adversary then charges you with the peak that makes you look worst. The adversary loses nothing by being allowed to charge weighted averages, $\max_i g_i^\top d = \max_{\alpha \in \Delta} (\sum_i \alpha_i g_i)^\top d$, where Δ is the set of weights $\alpha \ge 0$, $\sum \alpha_i = 1$ — an average never exceeds its largest member, and putting all weight on that member attains it. Your problem is $\min_d \max_\alpha$: you move first. The *dual* is the game with the order swapped, $\max_\alpha \min_d$. Moving second is an advantage, so the swapped value is $\le$ the original (weak duality); and for a payoff linear in each player over convex sets the advantage is worth nothing — the values are equal (von Neumann's minimax theorem, strong duality; this is the Lagrangian duality of §2 with d as primal and α as dual variable). Why swap? Because the swapped order is easy: for fixed α write $g(\alpha) = \sum \alpha_i g_i$; the inner problem $\min_{\|d\| \le 1} g(\alpha)^\top d$ is solved by pointing straight against the vector, $d = -g(\alpha)/\|g(\alpha)\|$, value $-\|g(\alpha)\|$. The outer problem is then

$$\min_{\alpha \in \Delta} \left\| \sum_i \alpha_i g_i \right\| \qquad d^* = \begin{cases} -g(\alpha^*)/\|g(\alpha^*)\| & \text{if the minimum norm is } > 0, \\ 0 & \text{if it is 0 (balance: no descent direction).} \end{cases}$$

Best direction = against the point of the gradients' convex hull closest to the origin; if that point is the origin, the peaks balance and α^* are the prices. **This is not steepest descent: it is the solution of a min–max problem, and it coincides with the negative gradient of one peak only when one peak is active.** (It is the steepest descent direction of the nonsmooth f in the Clarke sense; this note says "minimax direction".) Every "same swap of players" below is this paragraph again.

4.2 Solving the weight problem: the active-set walk

Squaring the norm changes nothing: minimise $q(\alpha) = \frac{1}{2}\alpha^\top H \alpha$ over Δ, $H_{ij} = g_i^\top g_j$ — a QP. The only difficulty is $\alpha \ge 0$: without it, "minimise a quadratic subject to $\sum \alpha = 1$" is one linear solve. Guess the support S (the peaks with nonzero weight):

$$\alpha_S = \frac{H_S^{-1}\mathbf{1}}{\mathbf{1}^\top H_S^{-1}\mathbf{1}}, \qquad \alpha_i = 0 \text{ off } S, \tag{$*$}$$

then verify: every weight nonnegative, and no excluded peak "wants in" (its reduced slope $(H\alpha)_j - \alpha^\top H \alpha$ is nonnegative). Naively, try all $2^m - 1$ supports. Practically, walk:

1. $S \leftarrow \{\text{tallest peak}\}$, $\alpha \leftarrow e_{i_{\max}}$. 2. $\hat{\alpha}_S \leftarrow H_S^{-1}\mathbf{1}/(\mathbf{1}^\top H_S^{-1}\mathbf{1})$, zero off S. 3. If $\hat{\alpha} \ge 0$: $\alpha \leftarrow \hat{\alpha}$, go to 4; else move from α toward $\hat{\alpha}$ until the first weight hits 0 ($t^\star = \min_{\hat{\alpha}_i < 0} \alpha_i/(\alpha_i - \hat{\alpha}_i)$), drop that index from S, go to 2. 4. Entry test $r_j = (H\alpha)_j - \alpha^\top H \alpha$ for $j \notin S$: if all $r_j \ge 0$ **stop** ($g^\star = \sum \alpha_i g_i$, $d = -g^\star/\|g^\star\|$ or 0); else add $\arg\min_j r_j$ to S, go to 2.

Each cycle is one small solve, q never increases, finitely many supports: it terminates in a handful of cycles. With prices p_i added to the objective ($\frac{1}{2}\alpha^\top H\alpha + p^\top\alpha$), the only change is $\alpha_S = H_S^{-1}(\mu\mathbf{1} - p_S)$ with $\mu = (1+\mathbf{1}^\top H_S^{-1}p_S)/(\mathbf{1}^\top H_S^{-1}\mathbf{1})$; the walk is identical. This walk, with these two tests, is what every QP in this note reduces to.

4.3 The step

With d^* in hand, an Armijo backtracking (§2.4) from a trial length chooses how far to go. File one point: the acceptance test evaluates the *true* maximum, so peaks with zero weight, which had no say in the direction, still veto the length. The full baseline method is: locate peaks → collect active gradients → weight problem → line search → repeat. It is correct and principled, and it is the seed of everything below.

5 The literature's four methods, each on its own terms

Each of the four is the baseline plus a few ingredients. They are presented first so that our method can then be read as a set of choices among known parts.

5.1 The descent line: Apkarian–Noll 2006 and hinfstruct

Ingredient 1: a finite, enlarged, recomputed frequency set. At each design the H_∞ norm is evaluated by the dense Hamiltonian level-set method (bisect on a level γ; γ is attained iff a certain $2n\times 2n$ Hamiltonian matrix has an eigenvalue on the imaginary axis; its axis eigenvalues bracket the intervals where the response exceeds γ; polish one peak per interval). The norm $f(\theta)$ being known, the axis is probed once more at the level $f-\varepsilon$, and the enlarged set I_ε collects one polished maximiser per interval above it — small ε: only the peaks tied with the maximum; larger ε: suboptimal local maxima too. Then (§VII.G of the paper) "a collection of logarithmically spaced frequencies" is added around the peaks, keeping those with $\sigma_1 \geq (1-\varepsilon_\omega)\|T\|_\infty$, $\varepsilon_\omega \in [0.05, 0.5]$, and the set is capped at the 50 largest. Around a peak ω_p the natural transcription is offsets in geometric progression, $\omega_p \pm \delta_0\rho^k$, so that the peak is bracketed at every scale (drift within an inner step has no preferred scale); a ladder of levels $\gamma(1-\varepsilon_k)$ with the crossing frequencies is an equally defensible reading, keyed to the peak's shape. No frequency persists as a *constraint*: the set is rebuilt from the norm evaluation at every design (the papers describe no carry-over; whether the implementations warm-start the peak search numerically from the previous design is unpublished, and would change nothing in the bookkeeping).

Ingredient 2: prices and a penalty instead of a ball. The direction problem becomes the *tangent program*

$$\min_d \max_{i\in I_\varepsilon} \big[\underbrace{f_i(\theta)-f(\theta)}_{\text{offset}\leq 0} + g_i^\top d\big] + \tfrac{1}{2}\|d\|^2 \quad\Longleftrightarrow\quad \min_{\alpha\in\Delta} \tfrac{1}{2}\Big\|\sum_i \alpha_i g_i\Big\|^2 + \sum_i \alpha_i\big(f-f_i\big), \qquad d = -\sum_i \alpha_i g_i,$$

by the same swap of players. Two changes against §4: the ball is replaced by $\frac{1}{2}\|d\|^2$, so the solution arrives with its own length (the trial step itself, then Armijo-backtracked); and each member's gap below the maximum, $f-f_i \geq 0$, enters as a *cost coefficient on its weight* in the dual objective — a cost, not a multiplier: the multipliers of this problem are the α_i themselves. The effect is precise: starting from a weight vector α, giving a little weight to peak i changes the dual objective by $(H\alpha)_i - \alpha^\top H\alpha + (f-f_i)$ per unit, so a sub-maximal peak enters the support exactly when its gap is smaller than the reduction of $\frac{1}{2}\|\sum\alpha_j g_j\|^2$ its gradient buys — when the gain in balancing outweighs the gap, i.e. when ignoring it would climb its peak fast. Exactly-active peaks have zero cost, and with all costs zero the plain weight problem returns. This moves the information about sub-maximal peaks from the line search (where the baseline had it: veto on the length) into the direction itself, which is what cures the baseline's zigzag.

Initialisation. A stabilising start is obtained by the same optimiser on a different objective: draw a random controller — pick a time scale ω_* (the band width or the least-damped mode's frequency) and a gain scale K_* (the inverse of the plant's peak gain); set the tridiagonal A_c with diagonal $-\omega_*|\xi|$ and off-diagonals $\omega_*\xi$, $\xi \sim \mathcal{N}(0,1)$, compute its r eigenvalues and redraw if any has $\mathrm{Re} \geq 0$ (a stable A_c is a convenience for the start, not a requirement afterwards); the output row $n \sim K_*\mathcal{N}(0,1)$, the gain $K \sim 0.1K_*\mathcal{N}(0,1)$, positions uniform in their ranges. HIFOO draws all entries unscaled; whether scaling matters is untested and is part of to-do item 1 — minimise the closed-loop spectral abscissa $\alpha(\theta) = \max_j \mathrm{Re}\,\lambda_j$ — nonsmooth, subgradients from eigenvectors — until $\alpha < -\varepsilon$, then switch to the norm. Burke–Lewis–Overton 2005; HIFOO 2006; A–N 2006 use it; hinfstruct/systune do the same in outline (`RandomStart`, preliminary stabilisation phase). A stable A_c is the sensible default for the draw (the loop then starts near the plant's own abscissa, not at a defect we planted), not a requirement.

Theory and tool. A–N 2006 prove accumulation at Clarke-stationary points. hinfstruct is this method plus engineering (multiple requirements in one max, systematic extended sets, multistart, the tridiagonal A_c parametrisation — a near-minimal, numerically tame canonical form, Apkarian–Dao–Noll 2015).

5.2 The nonsmooth quasi-Newton line: HIFOO

HIFOO (Burke, Henrion, Lewis & Overton 2006) minimises the H_∞ norm *directly*, as one nonsmooth function of the controller entries, evaluated densely (SLICOT's Hamiltonian norm) — no ε-set, no planes, no epigraph. The gradient it uses is the gradient of the single frequency the norm routine reports as attaining the maximum: away from ties that is the gradient of f; at a tie it is one of the Clarke subgradients (one branch), and the method relies on two things to cope — an inexact line search never lands exactly on a tie, so a gradient always exists (§2.5), and the finishing phases below examine several branches at once. Its engine, HANSO, runs three phases: (1) BFGS with an inexact Armijo–Wolfe line search on the nonsmooth norm — the behaviour of §2.5, "rather surprisingly" effective through kinks in the authors' words (HIFOO paper, §3); (2) a local *bundle* check that reuses the gradients already collected at the last few BFGS iterates — no new sampling — and asks whether some convex combination of them has small norm (the certificate of §4.1); (3) if not, *gradient sampling* proper, with fresh random points, to refine the point and return an optimality measure. Ties at the optimum — several frequencies attaining the norm, the generic outcome — are thus handled by (2)–(3): gradients taken at nearby points fall on different branches, and their hull reveals the balance.

Stability. Closed-loop stability is a constraint in HIFOO, enforced through the value of the objective: the norm routine checks the closed-loop eigenvalues first and returns $+\infty$ when any lies in the right half-plane — the H_∞ norm is defined only for stable loops, and the maximum of σ_1 over the axis, which is finite for an unstable loop, is not what is computed — so no line search ever accepts an unstable point. A stabilisation run (minimise the spectral abscissa from random or perturbed initial guesses) precedes the norm phase. What this cannot do is *follow* the boundary: an infinite-valued wall only rejects, it does not project, and H_∞ minimisers push closed-loop poles toward the axis. HIFOO's answer is a *barrier* option — a term added to the objective that grows as closed-loop eigenvalues approach the axis — which is a soft device, not a margin constraint: it keeps the poles away, it does not enforce $\alpha \leq -\varepsilon$, and the user tunes its weight. Requiring a margin is exactly the need HIFOO does not meet and GRANSO's constraints were added for (§5.4). Multistart is built in: several random initial controllers, and random perturbations of a user-supplied one. Its practical strengths are simplicity and robustness (phase 3 has convergence theory); its costs are the dense norm, a serial chain of thousands of evaluations, and no constraint handling beyond the infinite wall and the barrier. In the comparison of §6 it occupies the "exchange + SQP" column with the epigraph removed: one nonsmooth objective instead of many smooth constraints, curvature in phase 1 only, first-order finishing phases.

5.3 The bundle line: systune

Cutting planes. At a past design θ_k with value $f(\theta_k)$ and a (sub)gradient g_k, the tangent model $m_k(\theta) = f(\theta_k) + g_k^\top(\theta - \theta_k)$ lies below the graph of a convex function; the upper envelope of a collection of them, $m_{\mathscr{B}} = \max_k m_k$, is a piecewise-linear model built from first-order information alone — and if planes from both sides of a kink are present, their max reproduces the V: *the model sees the crease that any single gradient misses*. The collection is the *bundle*; in the refined variants each plane is a (design, frequency) pair — one plane per ε-active frequency of each visited design — so the frequency set is alive inside it, time-stamped.

Proximity control. A plane is trustworthy only near its point, so the trial point minimises the model plus a distance penalty to the best design so far, the *centre* θ_c: $\theta^+ = \arg\min m_{\mathscr{B}}(\theta) + \frac{1}{2\tau}\|\theta - \theta_c\|^2$. The quadratic term is distrust, not curvature — this is how a first-order method carries a quadratic term, and why classical bundle convergence is linear. The centre is the current iterate: it moves only to points that improved on it, so "centre" and "best design so far" are the same thing. Then reality: evaluate $f(\theta^+)$; if the actual decrease is at least a fraction of the predicted one, *serious step* (move the centre, enlarge τ), else *null step* (do not move, add the new plane — the model just learned why it was wrong — shrink τ). Why no line search along $\theta^+ - \theta_c$? Because in a nonsmooth setting that direction is a descent direction of the *model*, and there is no guarantee it is one of f: a line search cannot tell "wrong direction" from "too long a step", and would shrink uselessly along a bad ray, whereas a null step converts the failure into the plane that repairs the model. (Variants that add a line search still record the plane; the null step is the essential move.) Either serious steps drive f down or null steps make the model honest: for locally Lipschitz, lower-C^1 functions (a max of C^1 functions, ours away from singular-value coalescences), bounded below, every accumulation point of the serious steps is Clarke stationary (Noll, Prot & Rondepierre 2008; Noll 2010) — kinks included, no smoothness management.

The trial QP, derived. Put $\theta = \theta_c + d$ and refer each plane to the centre: $m_k(\theta_c + d) = f(\theta_c) - e_k + g_k^\top d$ with $e_k = f(\theta_c) - m_k(\theta_c) \geq 0$ (in the convex picture) the amount by which plane k sits below the true value at the centre. For nonconvex f a far-away plane may overestimate, so *downshift*: $e_k^+ = \max\{e_k,\ c\|\theta_k - \theta_c\|^2\}$ — by Taylor, if L_2 bounds the second derivatives on a ball around the centre, then $f(\theta) \geq m_k(\theta) - \frac{1}{2}L_2\|\theta - \theta_k\|^2$ on the ball, so lowering plane k by $c\|\theta_k - \theta_c\|^2$ with $c \geq \frac{1}{2}L_2$ makes it an underestimate near the centre; the algorithm never computes L_2: it fixes c and lets null steps discover where the model is inadequate. The trial problem is $\min_d \max_k[-e_k^+ + g_k^\top d] + \frac{1}{2\tau}\|d\|^2$; give the max its own name ζ (the epigraph of the *model*): $\min_{d,\zeta} \zeta + \frac{1}{2\tau}\|d\|^2$ s.t. $-e_k^+ + g_k^\top d \leq \zeta$ — a QP; swap the players and minimise in d in closed form ($d = -\tau\sum\alpha_k g_k$):

$$\boxed{\min_{\alpha\in\Delta}\ \tfrac{\tau}{2}\Big\|\sum_k \alpha_k g_k\Big\|^2 + \sum_k \alpha_k e_k^+, \qquad \theta^+ = \theta_c - \tau\sum_k \alpha_k g_k.}$$

The priced weight problem of §4.2 with planes for frequencies and downshifted errors for gaps. (In bundle–Newton variants the Euclidean proximity term becomes the B-norm $\frac{1}{2}d^\top Bd$ with a curvature estimate B — the same substitution that turns the tangent program into our model problem, §7.) *Aggregation*: the plane with slope $\sum\alpha_k g_k$ and error $\sum\alpha_k e_k^+$ is the model's supporting plane at the trial point; it may replace all planes with $\alpha_k > 0$ without changing the next solution (Kiwiel 1985), which bounds the bundle. *Constraints*: the progress function of §3 — minimise $\max\{f - \eta, g\}$, bisect on η; each η is one bundle run, warm-started, a $\times$5–10 multiplier on total work rather than a loss of rate, chosen over the exact penalty (which a bundle could minimise just as well) for its scale normalisation across requirements.

What is public about systune (Apkarian–Gahinet–Buhr 2014; Apkarian–Noll 2019 survey): objective = max over requirements and models; proximity-control bundle; planes over the extended frequency set of each trial (parallelisable in principle, serial as shipped); norms by the dense Hamiltonian method ($O(n^3)$); the progress function for hard constraints. hinfstruct descends from the descent line, systune from the bundle line; both live in the same toolbox.

5.4 The BFGS–SQP line: GRANSO

GRANSO (Curtis, Mitchell & Overton 2017) solves the generic problem $\min f(x)$ s.t. $c_i(x) \le 0$ with f, c_i possibly nonsmooth; the user supplies values and gradients. *Lineage.* GRANSO is HANSO's BFGS phase (§5.2) extended to constraints, by the same authors eleven years later: the exact penalty and steering replace HIFOO's infinite wall and barrier, so that margins and other inequalities can be imposed as constraints with multipliers. The two finishing phases were dropped: in the authors' benchmarks (Curtis, Mitchell & Overton 2017) BFGS–SQP alone was as reliable as versions with gradient sampling at a fraction of the cost — gradient sampling needs $m > n_\theta$ gradients per iteration, prohibitive for us — and the optimality certificate it provided is replaced by the solver's own stationarity measure. PyGRANSO is the Python port of GRANSO, with the same design: no gradient sampling. The authors' own control examples hand it the nonsmooth norm directly, evaluated densely, as objective or constraint — one function, kinks included, no epigraph; that is the intended use, HIFOO's approach with the newer engine. It works for them because their problems are small: a dense norm costs milliseconds, thousands of evaluations are affordable, a restart is cheap. The epigraph formulation is ours; the reason it was adopted is recorded, with the history, in §8. It knows nothing about frequencies, and it need not: *we* hand it the epigraph problem of §7.1 — variables (θ, t), objective t, and one constraint $c_i = \sigma_1(\omega_i;\theta) - t \le 0$ per frequency of Ω, each re-evaluated by the oracle at every iterate. The frequencies are the constraints; the tight ones are the active set of its QP; "tracking the peaks" is nothing more than re-evaluating the rows. Its own slack variables (§3) are a second, internal use of the epigraph idea, one per row, to make the penalised QP smooth. Everything it decides is decided on the exact penalty $\phi_\mu = \mu f + \sum[c_i]_+$ (§3) — and therefore so does our shipped inner solve, which is PyGRANSO: the exact penalty with steering is the constraint device of the frozen pipeline, inherited from the solver, not chosen among the four: the direction is the slack QP with quasi-Newton B (BFGS on $\nabla\phi_\mu$), μ is driven by steering, the step by a weak-Wolfe line search on ϕ_μ from $\beta = 1$ (§2.4) — so every constraint's violation, hence every frequency in Ω, vetoes a length — and iterates may be infeasible along the way. An optional trust region (a cap on $\|d\|$ in the QP) exists; the campaign measured it harmful and turned it off. What it lacks: for nonsmooth f or c_i the authors state no convergence guarantee; BFGS across a kink of a constraint absorbs a fictitious curvature, the line search shortens steps until progress stops, and the solver returns at its own tolerance — silently. The trade against the Apkarian lines: curvature (and a superlinear endgame) in exchange for theory.

6 The three methods side by side

(HIFOO is the "exchange + SQP" column with the epigraph removed, §5.2, and is not repeated.) Every method above is one loop — (i) choose the linear pieces to look at, (ii) solve a small QP for a step, (iii) control the step, (iv) consult the oracle, (v) update the pieces, and, outside, (vi) certify and enrich — with different fillings:

functionality	descent line	bundle line	exchange + SQP (ours)
linear pieces	gradients of the ε-active frequencies at the current design (rebuilt each step)	planes at past designs $\times$ their ε-active frequencies (accumulated, downshifted, aggregated)	gradients of all of a persistent set Ω at the current design
price of a piece	gap below the max, $f - f_i$	downshifted error e_k^+	slack below the level, $t - f_i$
metric of the step	Euclidean $\frac{1}{2}\|d\|^2$	Euclidean $\frac{1}{2\tau}\|d\|^2$	curvature $\frac{1}{2}d^\top Bd$
step control	Armijo on true f	serious/null, τ adapted	Wolfe on ϕ_μ; outer governor
kink between frequencies	priced hull	planes	active set
kink inside a frequency ($\sigma_1{=}\sigma_2$)	native (planes of both branches)	native	needs σ_2-enrollment (managed)
memory	no persistent constraint set (the papers describe none; internals unpublished)	the bundle	Ω (pieces) + B (curvature)
constraints	Armijo on a composite max	progress function, η-bisection	rows in the QP + guard + certificate
global certification	Hamiltonian (dense, per step)	Hamiltonian (dense, per trial)	disk certificate (sparse, per outer round)
inner/outer	one loop	one loop	explicit inner on Ω, outer certify/enrich
terminal rate	linear	linear (bundle–Newton: superlinear)	superlinear on a smooth active set
theory	Clarke (A–N 2006)	lower-C^1 (Noll et al.)	only after enrollment
warm start on the boundary	natural	natural	penalty: natural; IPM: awkward

Descent line. repeat: (i) norm at θ (Hamiltonian) $\to f, I_\varepsilon, g_i$; (ii) priced weight QP, $d = -\sum \alpha_i g_i$; (iii) Armijo from $\beta = 1$; (iv) $\theta \leftarrow \theta + \beta d$; (v) nothing kept.

Bundle line. centre θ_c, bundle $\mathscr{B}$; repeat: (ii) bundle QP, $\theta^+ = \theta_c - \tau\sum \alpha_k g_k$; (iv) $f(\theta^+)$ and its ε-active gradients; (iii) serious ($\theta_c \leftarrow \theta^+$, $\tau\uparrow$) or null ($\tau\downarrow$); (v) add planes, recompute and downshift e_k, aggregate if full. [Constraints: wrap in the η-bisection.]

Exchange + SQP. $\Omega \leftarrow$ starter grid $\cup$ seed peaks; outer repeat: inner repeat: (i) values and gradients on all of Ω (parallel, warm-started) + margin rows if flagged; (ii) epigraph QP with slacks and μ-steering; (iii) Wolfe line search on ϕ_μ; (iv) update (θ, t), BFGS on B, guard; until inner tolerance; (vi) certify the band, add located peaks and refuted points to Ω; until nothing new.

The two QPs face to face. Both are instances of one ancestor — minimise a level ζ plus a quadratic penalty on the step, subject to every linear piece staying below the level: $\min_{d,\zeta} \zeta + \frac{1}{2}d^\top Qd$ s.t. $a_k^\top d - p_k \le \zeta$, whose dual is $\min_{\alpha\in\Delta} \frac{1}{2}\|\sum \alpha_k a_k\|^2_{Q^{-1}} + \sum \alpha_k p_k$. Ours: pieces = frequencies in Ω at the current θ, $a_k = g_i$, price $t - f_i$, $Q = B$, $\zeta = \delta t$ — the epigraph of the *true function*. Bundle: pieces = planes at past designs, $a_k = g_k$, price e_k^+, $Q = \tau^{-1}I$ — the epigraph of the *model*. Descent line: pieces = ε-active frequencies, price $f - f_i$, $Q = I$. Same dual, same active-set walk; the differences are *which* linear information is on the table and *how* the step is scaled. Both methods handle nonsmoothness: the bundle by *imitating* the kink in one nonsmooth model, we by *enumerating* it in the active set of many smooth constraints. What neither may do is hand a kinked function to a smooth engine — which is exactly what the un-enrolled status quo does (§7.5).

7 Our method, as shipped

7.1 The epigraph reformulation: give the maximum its own name

On the finite list Ω, $\min_\theta \max_i f_{\omega_i}(\theta)$ is the same problem as

$$\min_{\theta,t} \; t \qquad \text{s.t.} \qquad f_{\omega_i}(\theta) \le t \;\; \forall\, \omega_i \in \Omega$$

(for fixed θ the best t is the maximum). The kink has left the objective — now the linear function t — and moved into bookkeeping: near the solution several smooth constraints are tight at once, and constrained optimisation is built for exactly that. In practice t is a genuine unknown stored next to θ (the solver's variable is $z = (\theta, t)$), sitting at or a hair above the tallest constraint, dipping below some f_{ω_i} during iterations (an infeasible point the penalty repairs), and equal to the achieved maximum over Ω at inner convergence — the candidate level γ, then certified. What the reformulation *buys* is a well-behaved Lagrangian, $\mathscr{L} = t + \sum_i \lambda_i (f_{\omega_i} - t)$: a smooth weighted sum of smooth pieces, the object every piece of constrained machinery is built around. Its KKT conditions, block by block, are $\sum_i \lambda_i g_i = 0$ (θ) and $\sum_i \lambda_i = 1$ (t): the balance conditions of §4.1 with the weights α renamed multipliers λ. The difference is who computes them — the baseline solves the weight problem by hand at every step; here a constrained solver finds the multipliers as a by-product, with curvature, with memory, without a hand-chosen trial length for the line search (the model supplies the scale, §2.4).

7.2 The model problem, and where every term comes from

Differentiate the Lagrangian twice in $x = (\theta, t)$: $\nabla_x \mathscr{L} = [\sum \lambda_i g_i;\; 1 - \sum \lambda_i]$ and $\nabla^2_{xx} \mathscr{L} = \operatorname{blkdiag}\big(\sum_i \lambda_i \nabla^2 f_{\omega_i},\, 0\big)$: the Lagrangian is *linear in* t, so there is no δt^2 term and no θ–t term, and the only curvature is $B := \sum \lambda_i \nabla^2 f_{\omega_i}$, in θ alone; the objective t has gradient $(0, 1)$, so there is no linear term in $\delta\theta$. The SQP model problem of §2.2 is therefore

$$\boxed{\min_{\delta\theta,\delta t} \; \delta t + \tfrac{1}{2}\,\delta\theta^\top B\,\delta\theta \qquad \text{s.t.} \qquad f_{\omega_i} + g_i^\top \delta\theta \le t + \delta t \;\; (\omega_i \in \Omega) \;\; + \text{other rows.}}$$

Derived, not proposed; B is the BFGS estimate of the multiplier-weighted curvature of the constraints (the objective is linear), an $n_\theta \times n_\theta$ matrix whose *dimension* is that of θ, not of the constraint list — enrolling a new row changes its size not at all and its *content* only gradually, as the secant updates re-weight the estimate toward the new multipliers. The same fact from the constraint side: "min t s.t. $f_i + g_i^\top d + \frac{1}{2} d^\top G_i d \le t$" has the same KKT points at the true multipliers, with the per-constraint curvatures collapsing into the single price-weighted B in the objective (Lemaréchal 1978; Kiwiel 1985; Lukšan–Vlček 1998). And $\frac{1}{2} d^\top B d$ read as a proximal term is the tangent program's $\frac{1}{2}\|d\|^2$ in the B-metric — the bridge to the descent line: *same priced program, curvature metric instead of Euclidean, epigraph variable so that a generic NLP solver can run it.*

Walk-through, one step from tied peaks. Guess the tight set $\mathscr{A}$ (m rows, gradients stacked as the rows of $G \in \mathbb{R}^{m \times n_\theta}$, gaps $r_i = f_i - t \le 0$). Stationarity in $\delta\theta$: $\delta\theta = -B^{-1} \sum \lambda_i g_i$; in δt: $\sum \lambda_i = 1$ — the balance, bent through B^{-1}. Substituting into the equalities, with $H = G B^{-1} G^\top$:

$$\begin{bmatrix} H & \mathbf{1} \\ \mathbf{1}^\top & 0 \end{bmatrix} \begin{bmatrix} \lambda \\ \delta t \end{bmatrix} = \begin{bmatrix} r \\ 1 \end{bmatrix} \quad \Longrightarrow \quad \delta t = \frac{\mathbf{1}^\top H^{-1} r - 1}{\mathbf{1}^\top H^{-1} \mathbf{1}}, \qquad \lambda = H^{-1}(r - \delta t\, \mathbf{1}).$$

Sanity check with $r = 0$, $B = I$: $\lambda = H^{-1}\mathbf{1}/(\mathbf{1}^\top H^{-1}\mathbf{1})$ — formula ($*$) symbol for symbol — and $\delta t = -\|\sum \lambda_i g_i\|^2$: *the epigraph machinery, run one step from tied peaks with unit curvature, is the Apkarian–Noll step.* Then the two tests of §4.2 (a negative multiplier: drop the row; a trampled outsider: add it) and repeat; GRANSO delegates this loop to its QP solver. Everything the constrained solver adds is in two symbols: B (curvature, replacing the unit-length rule) and r (slack, letting sub-maximal peaks participate — the prices of the descent line under another name).

7.3 The outer loop: certify and exchange

Optimise on a fixed grid and the solver digs the response down at the grid points while a peak grows between them; a locally convergent peak-finder (v6) tracks one peak while another becomes global. The cure is *certification*: after each inner solve, ask the oracle a question with a guaranteed answer — "over the whole band, is the response anywhere above t, and where?" — and act on it. The sweep, concretely: at a test frequency ω_0 one shift-invert eigensolve on the algebra note's pencil returns the nearest pencil eigenvalue at distance r_0; if $r_0 > 0$ the interval $(\omega_0 - r_0, \omega_0 + r_0)$ is certified below the level; if $r_0 \approx 0$ the level is attained nearby — a *refuted point*, polished into a peak. Test points are placed sequentially at the edge of the union of certified disks, $O(10^2)$ per sweep, finite by construction; one eigenvalue per shift is needed and one is requested by default, with escalation to a few when the residual is loose or the disk is critical. Known peaks are not re-located: they are re-polished by Newton's method on $\partial_\omega \sigma_1 = 0$ (first derivative from one inner product, second by differencing) from their previous positions (2–4 calls each), the incumbent level is their maximum, and the sweep certifies the gaps at $\gamma(1 + \mathrm{rtol})$ — coarse rtol (10^{-4}) inside the loop, 10^{-6} at the final certification.

Outer loop. 1. Inner solve on Ω (quasi-Newton constrained solver, warm-started). 2. Guard each accepted step (§7.4). 3. Certify the candidate over the band; add every located (near-)active frequency and refuted point to Ω. 4. Stop when certification adds nothing and the inner reports no progress — recording that the second condition means "inner at its limit", not "optimal". **Cadence of certificates:** peak certification every outer round; line sweep and continuation gate every accepted step; winding count (and, at benchmark scale, a dense eigensolve) at every save.

Working-set policy, as frozen. Ω opens with a uniform grid over the band (≈ 150 points) plus the seed's peaks; every outer round adds the certified peaks within the certification tolerance of the maximum and any refuted points; *nothing is removed*; every member's value and gradient are recomputed, in parallel and warm-started, at every inner iteration — $N = |\Omega|$ independent oracle calls, which is the shape a parallel machine wants and the reason the expensive unit of work is parallel here. This accounting changes if the oracle is the projected one of the algebra note (§10 there): the reduced model serves every member of Ω — values, σ_2, gradients — from 5–10 anchor factorisations made once per mesh, so the inner iteration needs no factorisation at all, the cost of a member of Ω drops to a few dozen sparse matvecs, and the case for retiring members becomes one of tidiness rather than of cost. What remains expensive is the certification, once per outer round, on the full model; that is where the parallel machine is then used. Runs end with $|\Omega| \approx 90$–170, of which 2–10 are tight. Against Apkarian–Noll's cap of 50, that is too many for the cavity, where each member costs a factorisation per iteration: the next iteration retires the starter grid after the first certified round and any member sitting below $(1 - \varepsilon_\omega)\gamma$ for two outer rounds, seeds level-filtered satellites (the A–N recipe) once per new peak and re-seeds only the currently active ones, and targets $|\Omega| \lesssim 50$ — certification makes retirement safe, since a wrongly retired frequency is re-located and re-added. Sub-maximal peaks influence the step through their slack (channel 2, the direction) and through the line search on ϕ_μ (channel 1, the length): both mechanisms of the literature are present; what enrichment adds is frequencies that were in neither.

7.4 Stability: a guard, a certificate, and the fix

The frequency response of an unstable loop is a finite, smooth function, so the inner problem does not see a crossing of the margin line; an optimiser sails across it in pursuit of a smaller peak (eight such designs were saved by an early pipeline, caught by an independent check). As shipped, stability enters the inner problem only while flagged: $\min t$ s.t. $\sigma_1(\omega_i; \theta) \le t$, $|\mathrm{Im}\,\lambda_k(A_c)| \le \omega_{\max}$, and $\mathrm{Re}\,\lambda_j(\theta) \le -\varepsilon$ for tracked j while flagged. Vocabulary, then mechanism. *Watch list* $\mathscr{W}$: the eigenvalues found near the margin line by a sweep of certified disks along $\mathrm{Re}\,s = -\varepsilon$ after each accepted step (which also proves nothing lies *on* the line). *Gate*: for each $\lambda_j \in \mathscr{W}$, continuation along the segment $\theta_k + s(\theta^+ - \theta_k)$ by shift-invert at its last position with adaptive Δs and a midpoint check (*homotopy split* when it fails); any $\mathrm{Re}\,\lambda_j(1) > -\varepsilon$

rejects the candidate. (Endpoint-only checks were the pre-gate design and failed by eigenvalue *identity*: the nearest eigenvalue to an old shift can be the wrong one after a swap.) *Governor*: a cap ρ on the length of an *accepted outer step*, i.e. on $\|\theta^+ - \theta_k\|$ between two verified designs — not on the inner solver's iterations, which keep their line search; the two act at different levels, the governor above the inner solve, the line search inside it. It is a trust region in the accept/reject sense (§2.4), but its radius is tied not to the validity of a local model of f but to the validity of the gate's premise (small steps keep the eigenvalue tracking unambiguous); it starts at $\rho = \infty$ — off until the gate produces evidence — is quartered on a rejection, halved on a split, and relaxed after clean acceptances. *Surrogate row*: when a watched eigenvalue enters the *watch band* $-\varepsilon - \delta_w < \operatorname{Re}\lambda_j \le -\varepsilon$, the constraint $\operatorname{Re}\lambda_j(\theta) + \varepsilon \le 0$ with gradient $\operatorname{Re}[w_j^{\mathsf{H}}(\partial_\theta A_{\mathrm{cl}})v_j/(w_j^{\mathsf{H}}M_{\mathrm{cl}}v_j)]$ (one eigenpair) is added to the solver's list; an active constraint inside the QP makes the step tangent to the boundary by itself — the "projected gradient", curvature included. *Layers*: gate (per step, conditional on tracking) and winding certificate (per save, an unconditional count right of $\operatorname{Re}s = -\varepsilon/2$) rest on different assumptions, which is why one caught a bug in the other. *What a rejection guard cannot do*: slide along the boundary; K+B+C converged pinned at a provably non-optimal boundary point and was unstuck by hand (worth 0.9%). The structural fix is to put the margin *inside the QP* as permanent per-eigenvalue rows for every near-boundary eigenvalue: an active row makes the QP step tangent to the boundary by construction (the projection you would otherwise do by hand), ties become several simultaneously active smooth constraints, and the subdifferential of the max is never formed — the first item of the next campaign. Controller poles: only the Im-cap is enforced, a heuristic proxy for "no closed-loop dynamics outside the certified band"; the rigorous device is the band-safety certificate, and a closed-loop bandwidth intent belongs in a frequency-weighted objective.

7.5 Residual kinks: where the reformulation itself fails

Two kinks must be told apart. *Between frequencies*: several peaks tie — the epigraph dissolved it (each is its own smooth constraint). *Inside a frequency*: $\sigma_1 = \sigma_2$ at one ω_i — the constraint function itself kinks, the epigraph never touched it (it was adopted to dissolve the first kind, which it did; the second kind was equally present in the direct use of PyGRANSO and in the descent line without branch enrichment — see §8), BFGS pairs straddling the jump encode a fictitious infinite curvature, and GRANSO stalls silently. This is not in tension with §2.5: there, the kink is in the *objective* being minimised, the ill-conditioned B is large *across* the crease, and the steps follow the crease — the curvature is fictitious but usefully oriented. Here the kink sits inside one *constraint* function while the objective t is linear; B estimates the Lagrangian's curvature for a QP whose constraints are supposed to be smooth tangents, the corrupted B has no crease to follow — the crease of the constraint is not a crease of the problem — and the QP step it produces is simply wrong, until the line search shrinks it to nothing. This is the measured signature of the cases that do not finish cleanly (K+B+C: a two-peak tie at a singular-value crossing, balance residual $\sim 10^4$, restarts buying 10^{-3} per round) — and the outer stopping rule cannot tell "solved" from "gave up", so a stall reads as convergence (the frozen pipeline labels such rows "inner at its limit" and measures their balance residuals; it does not cure them). The cure is the same medicine a third time: *name the members*. Enroll $\sigma_2(\omega_i;\theta) \le t$ as its own constraint — always at the active frequencies (one more Lanczos vector), thresholded with hysteresis elsewhere. Every branch is then smooth, B stays valid (no reset — resetting would pay the kink twice, once in the crossing and once in the amnesia), and standard SQP theory applies locally: convergence to a KKT point, superlinear once the active set settles. With enrollment early enough there is no remaining reason for the inner to stop short of optimality other than solver settings (QP accuracy, μ); the algorithmic risk that remains is only the timing of the detection. The residual difference from the bundle line is one word, *detection*: enrollment must notice the closing gap; the bundle needs no detector. Finally, a *Newton polish* of the balance system — unknowns $(\theta, t, \omega_1, \ldots, \omega_m, \alpha_1, \ldots, \alpha_m)$, equations $\sigma_1(\omega_i;\theta) = t$, $\partial_\omega \sigma_1(\omega_i;\theta) = 0$, $\sum \alpha_i \nabla_\theta \sigma_1 = 0$, $\sum \alpha_i = 1$ (Remez's equioscillation, solved by Newton) — gives a quadratic endgame once the membership is right.

8 Which method next? The choices, honestly

The common core any method needs to reach cavity scale: the sparse certified oracle; a finite working set with periodic certified enrichment (a bundle that evaluated true band maxima at every trial would spend everything on sweeps); parallel evaluation over that set. Why the core is non-negotiable: at cavity scale the unit of work is one sparse factorisation (minutes, gigabytes), a norm evaluation is hundreds of them, and the literature's methods — dense Hamiltonian norms, one trial design after another — are serial chains of such units, which no machine finishes; the literature does not address parallelisation at all, and any method we run must be shaped for it. Granted the core, the bundle line and the exchange share the same expensive primitive — values and gradients on Ω, in parallel, once per trial or per inner iteration — and are equally parallel at that granularity (the descent line's primitive is the bundle's). The only bundle that is *not* parallel is one that insists on a certified band-maximum at every trial, and that one is too slow for any purpose. With the projected oracle the primitive itself changes: the reduced model evaluates all of Ω without a factorisation, so the inner loop of *any* of the three methods — exchange, bundle on Ω, or a direct nonsmooth solver on Ω — runs on one node, and the parallel resource is spent on the two things that stay full-model: the anchors (once per mesh) and the certification sweep (per outer round, parallel over shifts). Two consequences for the choice ahead. The cost of an inner iteration no longer discriminates between the methods at all, so the decision rests entirely on the convergence differences listed above — kinks, constraints, rate, theory. And the cluster estimate for the cavity changes class: not fifty nodes held for every case, but one fat node for the optimisation and a handful borrowed for minutes per outer round — provided the reduced-subspace dimension in 2-D is what the GL numbers suggest, which is the first thing the algebra note's to-do list asks to measure.

The genuine differences: kinks (bundle native; SQP needs enrollment and its detector); constraints (SQP rows natural; bundle needs the progress function and its bisection); terminal rate (SQP superlinear on a settled active set; bundle linear unless bundle–Newton); software (IPOPT and active-set SQP mature; a nonconvex constrained bundle would be written by us — two hundred lines, now that the priced QP and the serious/null loop are on the table); warm starts (penalty and active-set: on the boundary for free; interior point: pushed off it, 5–15 iterations per exchange restart, against 10–30 restarts per case — bounded, not dominant, to be measured).

Two levels not to conflate (§2.3). Level 1 is the NLP method — the NLP being the epigraph problem of §7.1 itself, $\min t$ subject to its rows: an SQP loop (a sequence of QPs) or an interior point method applied to that problem as a whole (IPOPT — no QPs, one linear system per iteration); the exchange-restart count decides between them; the bundle is the third candidate. Level 2 exists only if level 1 is SQP: the QP solver inside each step — active set (qpOASES), interior point on the QP, or ADMM — and the frozen pipeline runs ADMM (OSQP inside PyGRANSO), whose step accuracy of a few digits is ample far from a solution and suspect at the terminal K+B+C iterates, where six nearly tied constraints are being balanced by the QP's multipliers.

Recommendation. Plan A: exchange + explicit constraints + enrollment + Newton polish, with the inner solver chosen by measurement between IPOPT and an active-set SQP (the walk-through QP, a warm-startable QP solver, a Wolfe line search — GRANSO without its nonsmooth ambitions), the 27-case frozen suite as regression test. Plan B: a proximity-control bundle on the same Ω with the progress function — the better-covered option at exactly the kinks where Plan A needs management, cheap to prototype, the designated fallback and a legitimate first choice for whoever prefers theory to software maturity. `systune` is excluded by two hard facts only (closed source cannot host our oracle; dense norms cannot reach cavity scale), not by architecture; `hinfstruct` has a norm certificate and convergence theory — it lacks scale, not rigour. *What was tried first, why it was abandoned, and a reassessment.* The study did not begin with the epigraph. Its first approach (report v2, §3.5 and §7) was PyGRANSO in its intended mode: BFGS–SQP applied directly to the nonsmooth $\gamma(\theta) = \max_{\omega\in\Omega} \sigma_1$ with the margin $\alpha(\theta) + \varepsilon \le 0$ as an explicit constraint — HIFOO's approach with GRANSO's engine, on the same finite set Ω and the same parallel oracle. Two things went wrong. First, scaling: with gradients of order 10–80 and useful steps of order 10^{-2}, BFGS's first step from $H_0 = I$ overshot into the unstable region

and several cases made zero progress; a calibrated H_0, then a fixed ladder of decreasing first steps, then an adaptive ladder that inferred the failure mode from each stage's outcome (improved: refine; stalled but feasible: enlarge; left the feasible set: shrink) were needed, tuned per case, and PyGRANSO's own automatic scalings failed. Second, the outcome on nineteen designs: nine reached a smooth stationary point (one active frequency), five an equalised optimum, and five stalled with a non-vanishing gradient and a runner-up peak 0.2–1.7% below the maximum — iterates creeping toward a tie between two frequencies, the line search rejecting every step that crossed the crease, the adaptive ladder unable to rescue them. The diagnosis recorded then: "a quasi-Newton line search is being asked to infer a kink structure from function values alone". The epigraph replaced it, and on the same cases the ties converged; the scaling problem disappeared with it, because a linear objective has no curvature to mis-scale ($H_0 = I$ suffices, no ladder).

Reassessed with the material of this note, that record says three things. (1) The five stalls were the kink-*between*-frequencies failure of §2.5: BFGS alone, without HANSO's bundle and sampling phases, approaching a tie from one branch — exactly the case Lewis–Overton describe as "works in practice, stalls silently when it fails". The epigraph cured it by making the tie an active set; that comparison, on nineteen designs with the same oracle and the same Ω, is the controlled experiment, and it was already run. (2) The v2 report described PyGRANSO as "BFGS–SQP with gradient sampling"; it has none (§5.4). The correction matters, because the phases that were missing are precisely the ones HIFOO uses to handle ties — so "direct nonsmooth mode" as tried was a truncated HIFOO, and its failure does not condemn the full method. (3) The reason for the epigraph is therefore neither parallelism nor certification: the direct mode ran on the same finite Ω with the same parallel evaluations, and the certificates belong to the outer loop, which is indifferent to the inner solver. The reason is inner convergence at ties, and nothing else; the earlier edition of this note overstated the structural argument. What follows for the to-do list: a retry of PyGRANSO in direct mode is not warranted — it would reproduce the v2 record; a retry of the *principled* direct nonsmooth approach is: either a HANSO-style three-phase engine, or the bundle inner of Plan B, which is the same idea with theory. Item 0b is revised accordingly. One residual defect of the v2 runs is worth remembering when reading them: a warm-restart bug in the real-constrained field cases (checkpoints not reproduced on resume) that was corrected later and may have contributed to two of the five stalls.

9 To-do list for the next code iteration

0. **QP-solver accuracy, first** — one line: tighten OSQP by three orders or swap in qpOASES, rerun the terminal K+B+C polish, read μ off the logs and add the μ-regrow detector (§3.2, §8). **0b.** No retry of PyGRANSO in direct mode (the v2 record is that experiment); instead, the principled direct-nonsmooth alternative — Plan B's bundle inner, or a HANSO-style three-phase engine — on the frozen suite, against the epigraph (§8). Also: correct the v2 report's statement that PyGRANSO uses gradient sampling.
1. **Random-start multistart with a stabilisation phase** (§5.1): scaled random draws, abscissa descent, then the norm; random starts *only* in a first study against Chen–Rowley seeds (HIFOO's unscaled recipe as the citable baseline, scaling as our variable). Keep the nesting lattice.
2. **Explicit stability constraints**: permanent per-eigenvalue rows for every tracked near-boundary eigenvalue; guard and certificates stay (§7.4). Cheap form, once the projected oracle exists: impose the margin on the *reduced* closed-loop pencil inside the inner loop (Mengi 2023, §6, shows the interpolation properties extend to a barrier on the reduced abscissa), gradients from reduced eigenvectors, winding count outside — algebra note, §10.6 there.
3. **σ_2-enrollment**: always at the active frequencies, thresholded with hysteresis elsewhere (§7.5).
4. **Neighbour seeding**: level-filtered log-spaced satellites per new peak (the A–N recipe; test both readings, ω-grid and level ladder) (§5.1).

5. **Retirement and $|\Omega| \lesssim 50$**: retire the starter grid after the first certified round and dormant members after two; validate at GL before the cavity (§7.3).
6. **Parallel gate**, validated by replaying the 27 frozen accepted-step chains bit for bit (same replay for the algebra note's parallel winding certificate).
7. **Controller-pole cap**: demote to an optional regulariser; band-safety certificate is the guarantee; closed-loop bandwidth via a frequency-weighted objective, in perspective.
8. **Inner solver, two levels**: prototype IPOPT and an active-set SQP, measure the restart overhead, let the number decide; give the SQP qpOASES or QPALM by item 0; bundle inner as Plan B.
9. **Newton polish** of the final balance system.
10. **Visualisation**: Ω, σ_1 on Ω, t, multipliers and spectrum per outer round (needs a small dump patch).
11. **Projected oracle in the inner loop** (algebra note, §10 there, and its to-do items): once the GL prototype reproduces the 27 certified optima, run the inner loop on the reduced model with certification as the enrichment rule, and re-measure the exchange-restart and warm-start costs of item 8 on that basis — the inner-iteration cost is then no longer the quantity that decides between IPOPT, an active-set SQP and the bundle inner. The loop itself — optimise on the reduced model, evaluate the full model at the reduced optimum, enrich there — is Algorithm 2.1 of Aliyev, Benner, Mengi & Voigt (2019), with a superlinear convergence theorem under a single-active-peak assumption; our version enriches at all of Ω (needed at balanced optima, where their assumption fails) and certifies (algebra note, §10.5 there). Two consequences for this list: (a) the random-start study of item 1 is free on the reduced model, since an evaluation costs microseconds; (b) for the position cases ($n_\theta = 2$–4) the reduced problem can be optimised *globally* (`eigopt`-style support functions, as they do) before certification — the first global answers this campaign could produce.
12. Remove the level scaling constant from the code.

References

- Apkarian & Noll, "Nonsmooth H_∞ synthesis", *IEEE TAC* 51 (2006) 71–86; multiband: *IEEE TAC* 52 (2007).
- Apkarian, Gahinet & Buhr, "Multi-model, multi-objective tuning of fixed-structure controllers", ECC 2014.
- Apkarian & Noll, "Structured H_∞ control of infinite-dimensional systems" / survey, 2018–2019.
- Apkarian, Dao & Noll, "Parametric robust structured control design", *IEEE TAC* 60 (2015).
- Gahinet & Apkarian, "Structured H_∞ synthesis in MATLAB", IFAC 2011.
- Burke, Lewis & Overton, "A robust gradient sampling algorithm for nonsmooth, nonconvex optimization", *SIAM J. Optim.* 15 (2005).
- Burke, Henrion, Lewis & Overton, "HIFOO — a MATLAB package for fixed-order controller design and H_∞ optimization", IFAC ROCOND 2006.
- Lewis & Overton, "Nonsmooth optimization via quasi-Newton methods", *Math. Program.* 141 (2013) 135–163.
- Curtis, Mitchell & Overton, "A BFGS-SQP method for nonsmooth, nonconvex, constrained optimization and its evaluation using relative minimization profiles", *Optim. Methods Softw.* 32 (2017) 148–181 (GRANSO).
- Byrd, Nocedal & Waltz, "Steering exact penalty methods for nonlinear programming", *Optim. Methods Softw.* 23 (2008).
- Han, "A globally convergent method for nonlinear programming", *J. Optim. Theory Appl.* 22 (1977); Wilson (1963), Powell (1978) — SQP as Newton on KKT.
- Nocedal & Wright, *Numerical Optimization*, 2nd ed., Springer 2006 (chs. 17–19).
- Wächter & Biegler, "On the implementation of an interior-point filter line-search algorithm", *Math. Program.* 106 (2006) (IPOPT).
- Wright, "Ill-conditioning and computational error in interior methods", *SIAM J. Optim.* 9 (1998).
- Stellato et al., "OSQP: an operator splitting solver for quadratic programs", *Math. Program. Comput.* 12 (2020); Ferreau et al., qpOASES, *Math. Program. Comput.* 6 (2014).

- Kiwiel, *Methods of Descent for Nondifferentiable Optimization*, Springer 1985; Lemaréchal, "Nonsmooth optimization and descent methods", IIASA 1978.
- Noll, Prot & Rondepierre, "A proximity control algorithm to minimize nonsmooth and nonconvex functions", *Pacific J. Optim.* 4 (2008); Noll, "Cutting plane oracles to minimize non-smooth non-convex functions", *Set-Valued Var. Anal.* 18 (2010).
- Lukšan & Vlček, "A bundle–Newton method for nonsmooth unconstrained minimization", *Math. Program.* 83 (1998).
- Hettich & Kortanek, "Semi-infinite programming: theory, methods, and applications", *SIAM Review* 35 (1993); Remez (1934).
- Aliyev, Benner, Mengi, Schwerdtner & Voigt, "Large-scale computation of L_∞-norms by a greedy subspace method", *SIAM J. Matrix Anal. Appl.* 38 (2017); Aliyev, Benner, Mengi & Voigt, "A subspace framework for H_∞-norm minimization", *SIAM J. Matrix Anal. Appl.* 41 (2020), arXiv:1905.04208; Mengi, "Large-scale estimation of dominant poles of a transfer function by an interpolatory framework", *SIAM J. Sci. Comput.* 44 (2022); Mengi, "A subspace framework for L_∞ model reduction", arXiv:2309.08011 (2023); Schwerdtner, Mengi & Voigt, "Certifying global optimality for the L_∞-norm computation of large-scale descriptor systems", *IFAC-PapersOnLine* 53(2) (2020) — the certificate of the oracle.
- Boyd & Vandenberghe, *Convex Optimization*, Cambridge 2004 (§4.1.3, §4.2); Parlett, *SIAM J. Matrix Anal. Appl.* 13 (1992) (tridiagonal realisations).

10 What to remember

Minimising a worst case means balancing peaks, and the balance point is a kink where plain calculus fails (§1). The balance can be computed by hand at each step — the weight problem, whose weights are the prices of the peaks that fight each other (§4) — or delegated to a constrained solver by giving the maximum its own variable, whereupon the weights reappear as multipliers (§7.1). The three methods of the literature and ours are one QP wearing four coats (§6); they differ in which linear pieces they keep and how they scale the step. The continuum of frequencies is handled by certify-and-exchange, whose expensive work is parallel (§7.3). Stability is, in the frozen pipeline, a guard rather than a constraint, with a measured limitation at the boundary (§7.4); the method's residual failures sit exactly where its smoothness assumptions break — singular-value crossings inside a frequency and the inner solver's silent stalls there (§7.5) — which is why the next campaign's first task is an inner solver that makes every kink it meets explicit.